KGiSL Institute of Technology (An Autonomous Institution)
Affiliated to Anna University, Approved by AICTE, Recognized by UGC, Accredited by NAAC & NBA (B.E-CSE,B.E-ECE, B.Tech-IT),
365, KGiSL Campus, Thudiyalur Road, Saravanampatti, Coimbatore – 641035.

24UCS271 – PROGRAMMING IN C

LAB RECORD

DEPARTMENT OF SCIENCE AND HUMANITIES

NAME : SEETHALAKSHMI. J

ROLL NO : 25UIT230

REG. NO : 7117 25UIT230

BRANCH & SEC : B.Tech IT & B

ACADEMIC YEAR & SEM : 2025-2026 & II

KGiSL Institute of Technology
(An Autonomous Institution)
Affiliated to Anna University, Approved by AICTE, Recognized by UGC,
Accredited by NAAC & NBA (B.E-CSE,B.E-ECE, B.Tech-IT),
365, KGiSL Campus, Thudiyalur Road, Saravanampatti, Coimbatore – 641035.

LABORATORY RECORD BOOK

Certified that, this is a bonafide record of practical work done by .....S.E.E.THALAKSHMI.J.....of.....B.Tech.IT-B.....
.....ENGINEERING branch in **PROGRAMMING IN C LABORATORY (24UCS271)** during II semester of the academic year 2025-2026 EVEN SEMESTER.

Model Practical Examination held on: 18/05/2026

Faculty in charge

LIST OF EXERCISES

S.No	Date	Excersises	Total Marks obtained (100)	CO	CO Marks	CO Marks obtained	Sign
1		I/O statements, operators, expressions		CO1	15	15	
2		Branching Statements: if, if-else, nested if, goto, Switch-case, break, Continue		CO1			
3		Looping Statements		CO1			
4		1D and 2D Arrays		CO2	15	14	
5		String Operations		CO2			
6		Functions: Passing Returning Values and Arrays		CO3	15	14	
7		Recursion		CO3			
8		Pointers: Pointers to functions, Arrays, Strings, Pointers to Pointers, Array of Pointers		CO3			
9		Structures and Unions: Nested Structures, Pointers to Structures, Arrays of Structures.		CO4	15	14	
10		File operations: reading and writing, file pointers, file operations, random access.		CO5	15	14	
ASSESSMENT OF EXPERIMENTS (75%)			CO1 – CO5		75	71	
MODEL EXAM (25%)			ALL CO'S		25	24	
TOTAL - (75% WA + 25% MODEL)					100	95	

Faculty in charge

Exp.No: 1	**Mark calculator**
Date:	

Aim:

To print the total and average of the marks

Procedure:

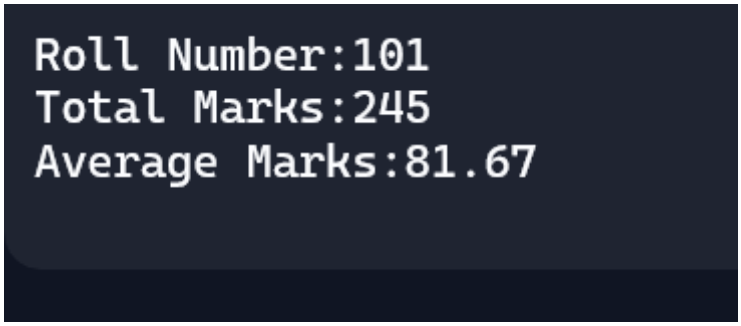
1. Assign the marks, roll number with 'int' datatype.
2. Get the roll number, three marks as input.
3. Calculate the total and average of the marks given and print it.

Program:

ers > KiTE > AppData > Local > Temp > 8e365008-7397-4f85-badb-7ec3cbbba127_W1 - In-Lab.zip.127 > C student.c > ...

```c
#include <stdio.h>

int main()
{
    int roll;
    int a,b,c;
    int total;
    float average;
    scanf("%d",&roll);
    scanf("%d %d %d",&a,&b,&c);
    total=a+b+c;
    average=total/3.0;
    printf("Roll Number:%d\n",roll);
    printf("Total Marks:%d\n",total);
    printf("Average Marks:%.2f\n",average);
    return 0;
}
```

Output:



Result:

Hence the total and average of the given marks is calculated.

Exp.No: 2	**Employee Performance evaluation system**
Date:	

Aim:

To develop employee performance evaluation system we need to store employee details, calculate performance scores, assign grades and display evaluation result .

Procedure:

1. Start the Program

2.create a structure containing employee id, performance grade, total score, grade

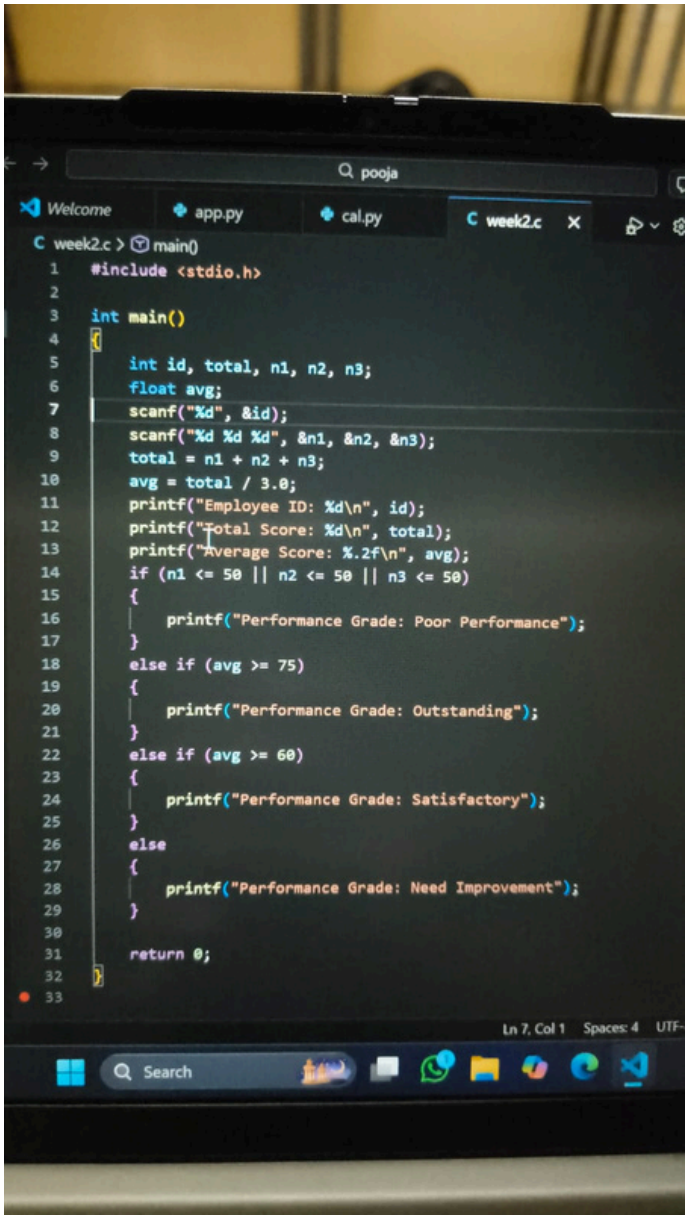
3.calculate total and average

4.assign grade for each performance

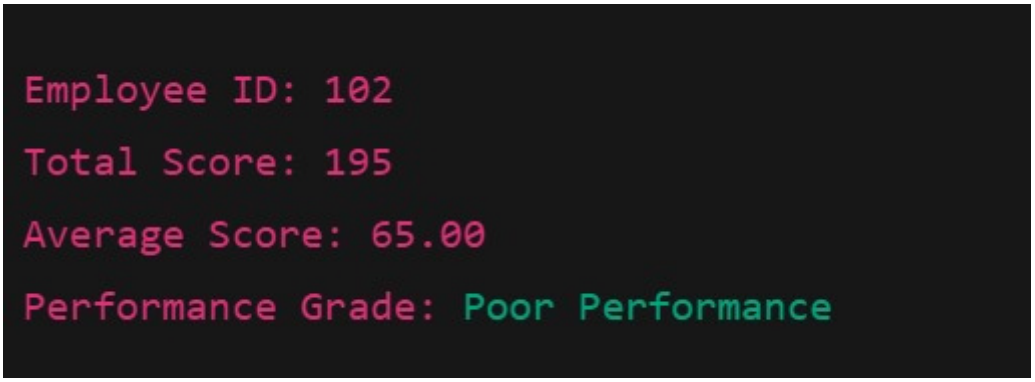
5.display result

6. End the program

Program:


pooja
Welcome
app.py
cal.py
week2.c
week2.c > main()
#include <stdio.h>
int main()
{
int id, total, n1, n2, n3;
float avg;
scanf("%d", &id);
scanf("%d %d %d", &n1, &n2, &n3);
total = n1 + n2 + n3;
avg = total / 3.0;
printf("Employee ID: %d\n", id);
printf("Total Score: %d\n", total);
printf("Average Score: %.2f\n", avg);
if (n1 <= 50 || n2 <= 50 || n3 <= 50)
{
printf("Performance Grade: Poor Performance");
}
else if (avg >= 75)
{
printf("Performance Grade: Outstanding");
}
else if (avg >= 60)
{
printf("Performance Grade: Satisfactory");
}
else
{
printf("Performance Grade: Need Improvement");
}
return 0;
}
Ln 7, Col 1
Spaces: 4
Search

Output:



Result:

The program has been run successfully and the output is displayed.

Exp.No: 3	FizzBuzz sequence generator
Date:	

Aim:

To print the sequence of outputs from 1 to the given number and to find whether each number satisfies the given conditions.

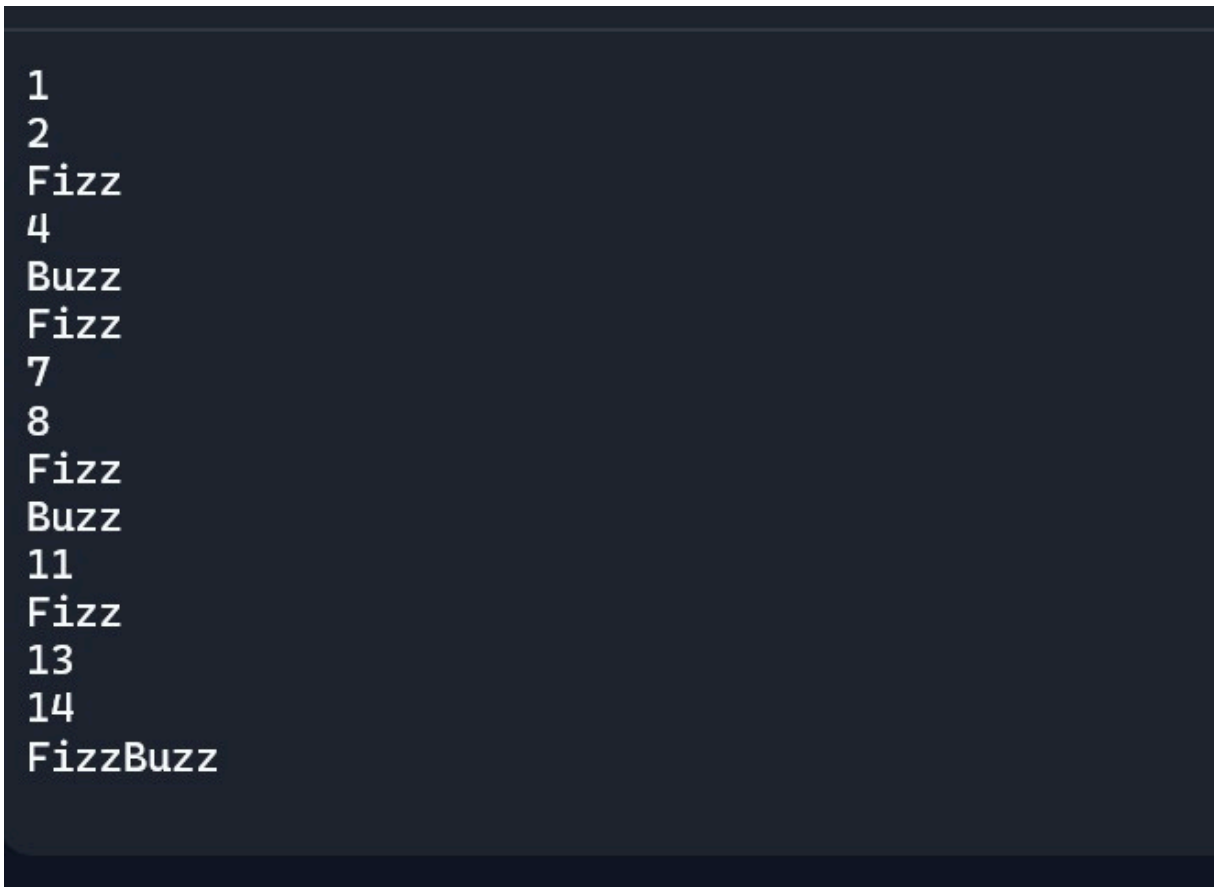
Procedure:

1. get an input that is the limit for our sequence.
2. Iterate through the numbers from 1 to the limit 'N'.
3. Check if the each number is divisible by 3 and 5 and print 'FizzBuzz'.
4. Check if the each number is divisible by 3 and print 'Fizz'.
5. Check if the each number is divisible by 5 and print 'Buzz'
6. If the number does not comm under any of the above condition then print the number itself.

Program:

```c
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    for(int i = 1; i <= N; i++) {
        if(i % 3 == 0 && i % 5 == 0) {
            printf("FizzBuzz\n");
        }
        else if(i % 3 == 0) {
            printf("Fizz\n");
        }
        else if(i % 5 == 0) {
            printf("Buzz\n");
        }
        else {
            printf("%d\n", i);
        }
    }

    return 0;
}
```

Output:



Result:

Thus we get the sequence of numbers with conditions.

Exp.No: 04	1D and 2D array
Date:	

Aim:

To write a C program on reading marks of students and calculating their total, average and display results.

Procedure:

Step 1 : Start the program.

Step 2 : Declare the required varibles like n, marks[n][3], i, j, avg, tot=0.

Step 3 : Read the no. of students.

Step 4 : Use nested loop to get input marks and store them in 2D array.

Step 5 : Use a loop to calculate total marks.

Step 6 : Print the total marks for each student.

Step 7 : For each subject initialize sum=0.

Step 8 : Use a loop to calculate the sum of marks of all students for that subject.

Step 9 : Compute average using avg=(float)sum/n and display the average marks of each subject .

Step 10: Stop the program.

Program:

```c
#include <stdio.h>

int main()
{
    int n,i,j;

    if (scanf("%d", &n) != 1)
        return 0;

    int marks[n][3];
    float avg;

    // Input marks
    for (i=0;i<n;i++)
    {
        for (j=0;j<3;j++)
        {
            scanf("%d",&marks[i][j]);
        }
    }

    // Calculate total for each student
    for (i=0;i<n;i++)
    {
        int tot=0;
        for (j=0;j<3;j++)
        {
            tot+=marks[i][j];
        }
        printf("Student %d Total: %d\n",i+1,tot);
    }

    printf("\n");

    // Calculate average for each subject
    for (j=0;j<3;j++)
    {
        int sum=0;
        for (i=0;i<n;i++)
        {
            sum+=marks[i][j];
        }
        avg=(float)sum/n;
        printf("Subject %d Average: %.2f\n",j+1,avg);
    }

    return 0;
}
```

Output:

```
3
78 85 69
90 88 92
65 70 72
Student 1 Total: 232
Student 2 Total: 270
Student 3 Total: 207

Subject 1 Average: 77.67
Subject 2 Average: 81.00
Subject 3 Average: 77.67
```

Result:

The given program is executed and successfully displayed.

Exp.No: 05	**Username processing system using string operation in c**
Date:	

Aim:

to develop a c program that read a user's first name and last name , determines their length, compare the two names, and generates a username by concatenating them using standard string handling functions

Procedure:

step 1: start the program

step 2: Declare two character arrays first and last to store the names.

step 3: Use scanf() to read the first name and last name from the user.

step 4: Use strlen() to calculate the length the length of both strings.

step 5: Display the length using printf()

step 6: Use strcmp() to compare the two strings().

step 7: If both are equal, print "Names are equal".

step 8: Use strcat() to concatenate the first name and last name.

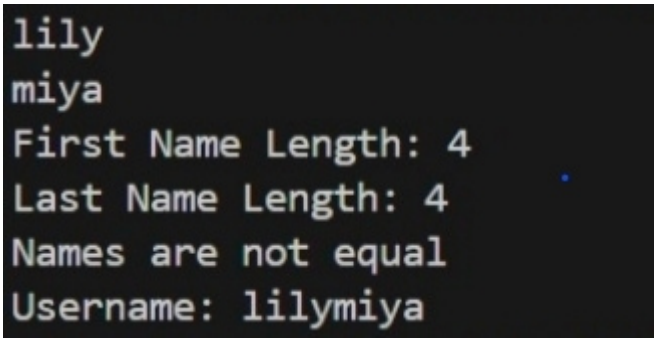
step 9: display the concatenated strings as the username.

step 10: end the program

Program:

```c
#include<stdio.h>
#include<string.h>
int main(){
    char first[100];
    char last[100];
    scanf("%s\n",first);
    scanf("%s",last);
    printf("First Name Length: %d\n",strlen(first));
    printf("Last Name Length: %d\n",strlen(last));
    if(strcmp(first,last)==0)
    {
        printf("Names are equal\n");
    }
    else
    {
        printf("Names are not equal\n");

    }
    printf("Username: %s",strcat(first,last));
    return 0;

}
```

Output:



Result:

the program successfully reads first and last name calculate their length compare the generates their username concatenate the both names

Exp.No: 06	**Program to find the prime factors of a given number using C.**
Date:	

Aim:

To write and execute a C program that finds all the prime factors of a given integer using a function.

Procedure:

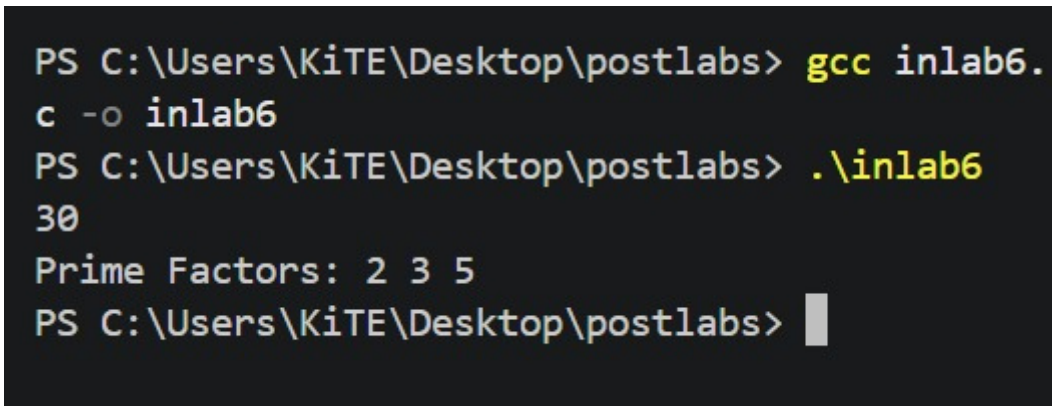
1. Start the program.
2. Include the standard input-output header file<stdio.h>.
3. Define a function prime_factors(int n, int fact[]) to calculate prime factors.
4. Initialize a counter variable c = 0 to store the number of factors.
5. Use a loop from i = 2 to n to check divisibility.
6. If divisible, store the factor, divide the number, and repeat the check.
8. Display the prime factors using a loop.
9. Stop the program.

Program:

inlab6.c

```c
#include <stdio.h>
int prime_factors(int n,int fact[])
{
    int c=0;
    for(int i=2;i<=n;i++)
    {
        if(n%i==0)
        {
            fact[c++]=i;
            n=n/i;
            i--;
        }
    }
    return c;
}
int main()
{
    int n,i;
    scanf("%d",&n);
    int fact[100];
    int c=prime_factors(n,fact);
    printf("Prime Factors: ");
    for(i=0;i<c;i++)
    {
        printf("%d",fact[i]);
        if(i!=c-1)
        {
            printf(" ");
        }
    }
    return 0;
}
```

Output:



Result:

Thus the C program to find the prime factors of a given number was successfully implemented and executed. The program correctly identifies and displays all prime factors of the input number.

Exp.No: 7	BINARY SEARCH USING RECURSION
Date:	

Aim:

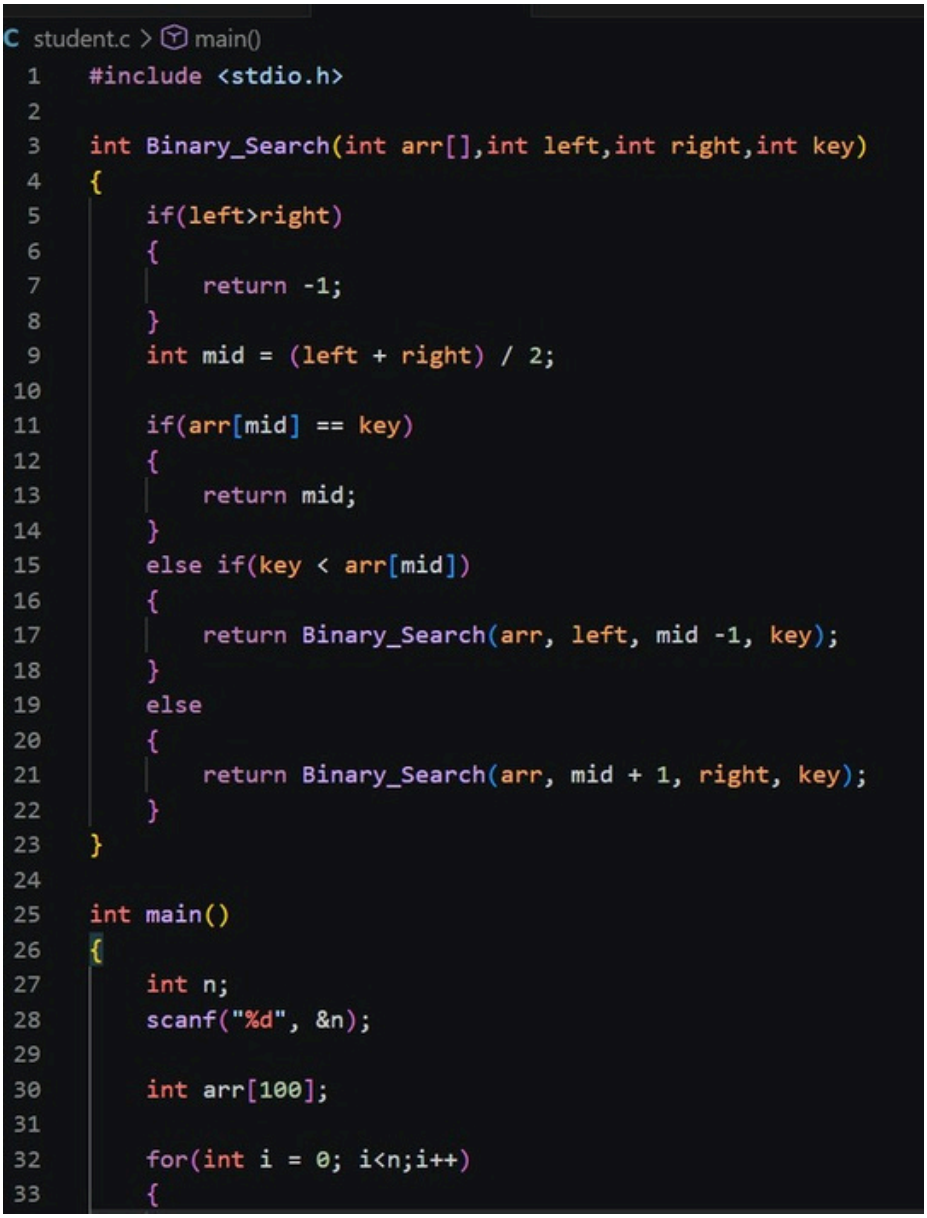
To write a C program that searches for a given element in a sorted list of elements using binary search technique
implemented recursively.

Procedure:

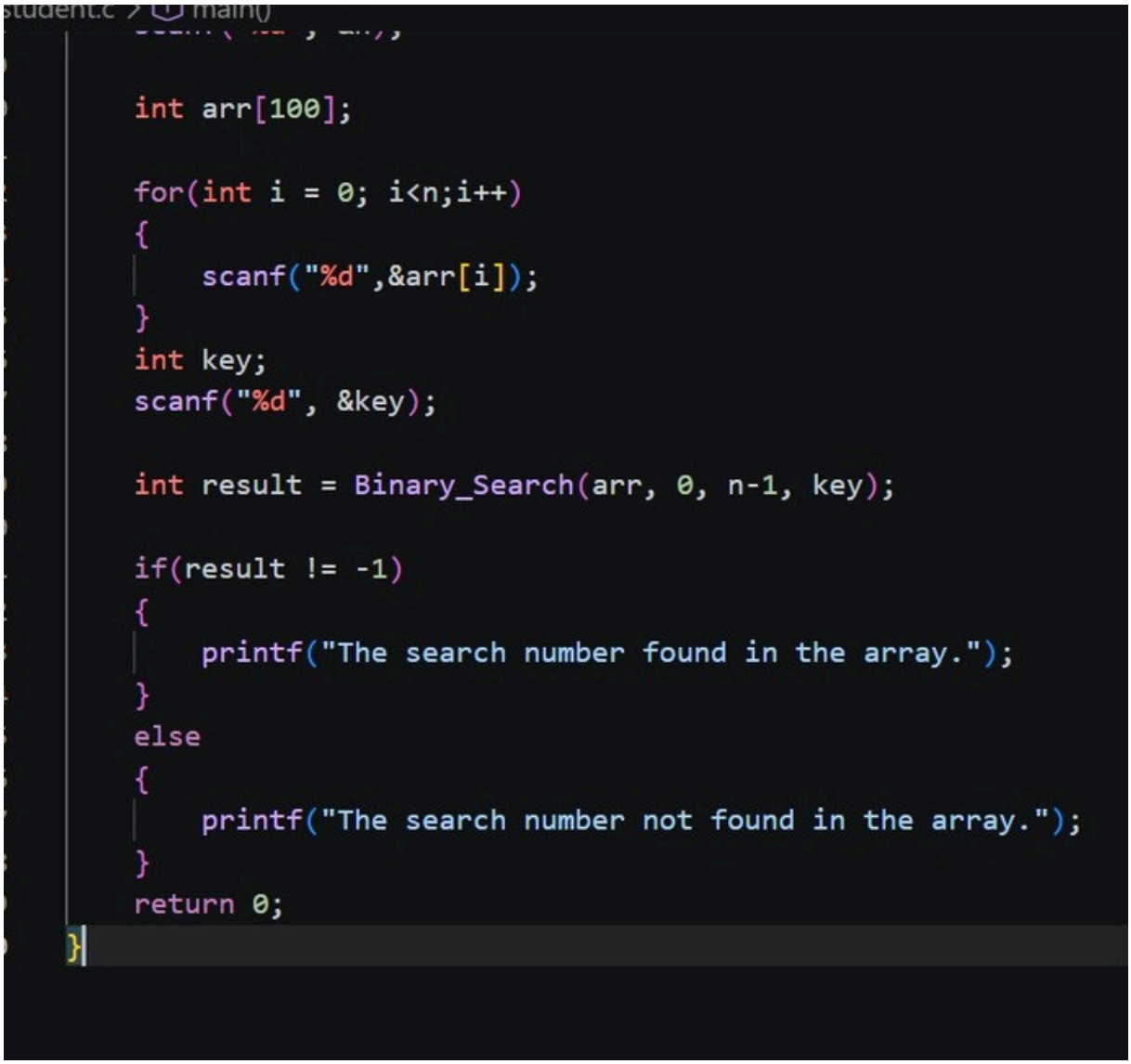
STEP 1 : Startthe program.

STEP 2 : Include the headerfile <stdio.h>. STEP 3 : Read an integer as an input from the user that represents the number of elements. STEP 4 : Read the integers in ascending order and store them in an array. STEP 5 : Read an integer as a key value to be searched. STEP 6 : Perform binary search using recursive function. STEP 7 : Determine whether the key exists or not. STEP 8 : Display the result. STEP 9 : End the program.

Program:

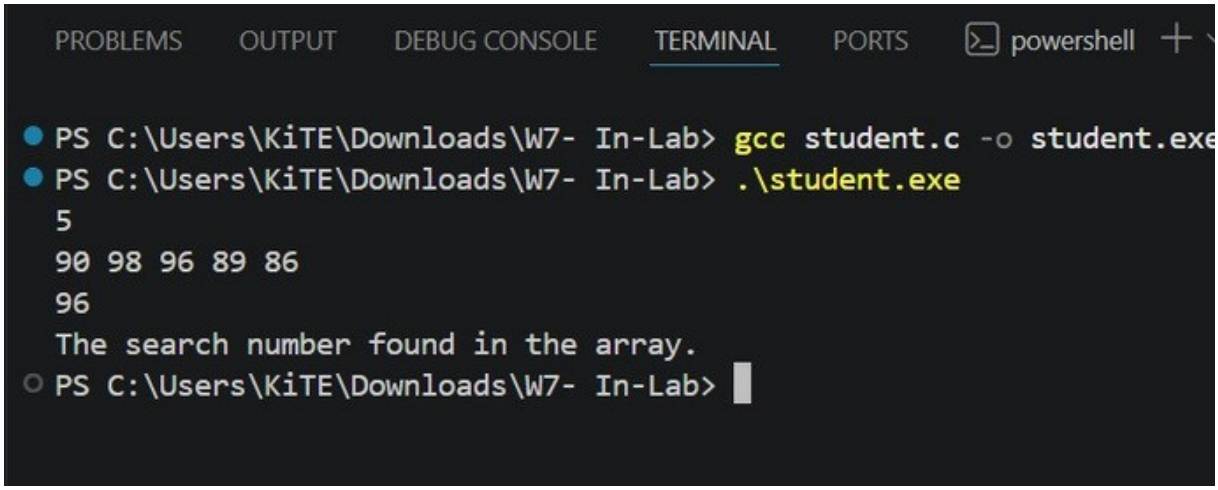


```c
#include <stdio.h>

int Binary_Search(int arr[],int left,int right,int key)
{
    if(left>right)
    {
        return -1;
    }
    int mid = (left + right) / 2;

    if(arr[mid] == key)
    {
        return mid;
    }
    else if(key < arr[mid])
    {
        return Binary_Search(arr, left, mid -1, key);
    }
    else
    {
        return Binary_Search(arr, mid + 1, right, key);
    }
}

int main()
{
    int n;
    scanf("%d", &n);

    int arr[100];

    for(int i = 0; i<n;i++)
    {
```



```c
int arr[100];

for(int i = 0; i<n;i++)
{
    scanf("%d",&arr[i]);
}
int key;
scanf("%d", &key);

int result = Binary_Search(arr, 0, n-1, key);

if(result != -1)
{
    printf("The search number found in the array.");
}
else
{
    printf("The search number not found in the array.");
}
return 0;
}
```

Output:



Result:

The above program is executed successfully and the output is verified.

Exp.No: 8	**Program to swap two elements in an array using a function in C**
Date:	

Aim:

To write and execute a C program that swaps two elements of an array using a function and pointers.

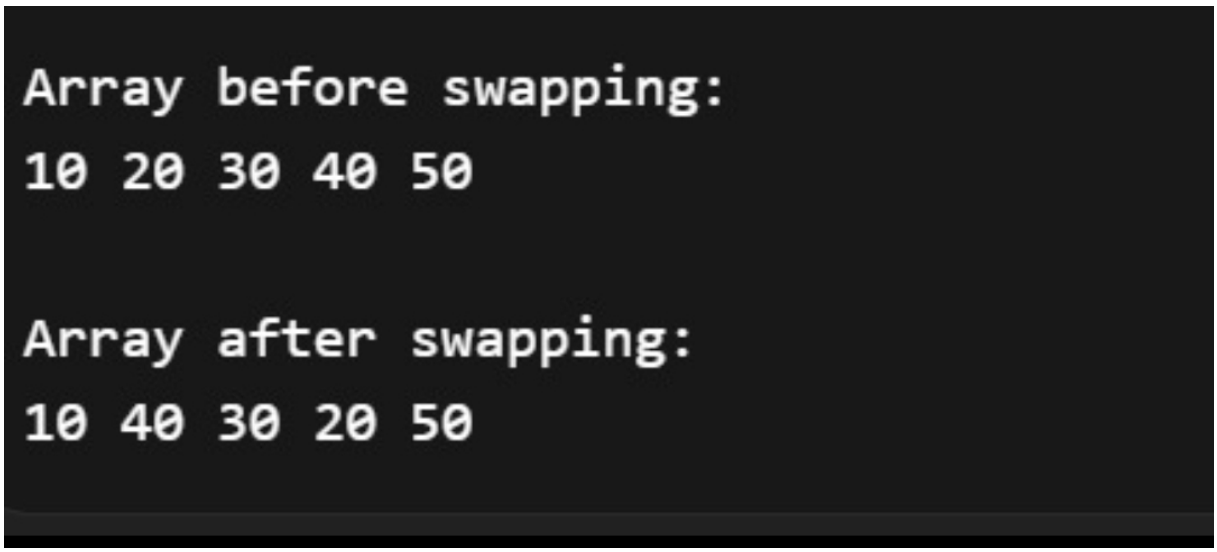
Procedure:

1. Start the program.
2. Include the standard input-output header file <stdio.h>.
3. Define a function swap(int *a, int *b) to interchange two numbers using pointers.
4. Inside the function, declare a temporary variable temp.
5. Store the value of *a in temp.
6. Assign the value of *b to *a.
7. Assign the value stored in temp to *b.
8. In the main() function, declare an integer variable N to store the size of the array.
9. Read the value of N from the user.
10. Declare an array arr[N].
11. Use a loop to read N elements into the array.
12. Read the two index positions i and j whose elements need to be swapped.
13. Display the array before swapping.
14. Call the swap() function by passing the addresses of arr[i] and arr[j].
15. Display the array after swapping.
16. Stop the program.

Program:

```c
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int N;
    scanf("%d", &N);
    int arr[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }
    int i, j;
    scanf("%d %d", &i, &j);
    printf("Array before swapping:\n");
    for (int k = 0; k < N; k++) {
        printf("%d", arr[k]);
        if (k < N - 1) printf(" ");
    }
    printf("\n\n");
    swap(&arr[i], &arr[j]);
    printf("Array after swapping:\n");
    for (int k = 0; k < N; k++) {
        printf("%d", arr[k]);
        if (k < N - 1) printf(" ");
    }
    return 0;
}
```

Output:


Array before swapping:
10 20 30 40 50
Array after swapping:
10 40 30 20 50

Result:

Hence two elements in an array are swamped.

Exp.No: 09	**Program to generate and display library book records using nested structure in C**
Date:	

Aim:

To write and execute a C program that generates and displays multiple library book records using nested structures.

Procedure:

1.Start the program.

2. Include the standard input-output header file stdio.h.

3. Define a structure Date with members day, month, and year.

4. Define another structure Book with members book_id, title, roll_no, student_name, and a nested structure issue_date of type Date.

5. In the main() function, declare an integer variable n to store the number of records.

6. Read the value of n using scanf().

7. Declare an array of structures records[n] to store multiple book records.

8. Use a for loop from i = 0 to n-1 to read the details of each record.

9. Inside the loop, read the book ID, title, roll number, student name, and issue date (day, month, year).

10. Store the issue date inside the nested structure issue_date.

11. Use another for loop to display all the entered records.

12. Print the book ID, title, student name with roll number, and issue date in proper format.

13. Stop the program.

Program:

```c
#include <stdio.h>
struct Date {
    int day;
    int month;
    int year;
};
struct Book {
    int book_id;
    char title[100];
    int roll_no;
    char student_name[100];
    struct Date issue_date;  // nested structure
};
int main() {
    int n;
    scanf("%d", &n);
    struct Book records[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &records[i].book_id);
        scanf("%s", records[i].title);
        scanf("%d", &records[i].roll_no);
        scanf("%s", records[i].student_name);
        scanf("%d %d %d",
              &records[i].issue_date.day,
              &records[i].issue_date.month,
              &records[i].issue_date.year);
    }
    for (int i = 0; i < n; i++) {
        printf("Book ID: %d\n", records[i].book_id);
        printf("Title: %s\n", records[i].title);
        printf("Issued To: %s (Roll No: %d)\n",
               records[i].student_name,
               records[i].roll_no);
        printf("Issue Date: %02d/%02d/%04d\n",
               records[i].issue_date.day,
               records[i].issue_date.month,
               records[i].issue_date.year);
        if (i != n - 1) {
            printf("\n");
        }
    }
    return 0;
}
```

Output:

```
Book ID: 101
Title: C_Programming
Issued To: Ravi (Roll No: 23)
Issue Date: 10/04/2026

Book ID: 102
Title: DataStructures
Issued To: Priya (Roll No: 24)
Issue Date: 11/04/2026
```

Result:

Thus, the C program to generate and display multiple library book records using nested structures was written, compiled, and executed successfully, and the output was verified.

Exp.No: 10	**Program to create, write, append, and read student details using file handling in C**
Date:	

Aim:

To write and execute a C program that stores student details in a file, appends additional information, and displays the file contents using file handling functions.

Procedure:

1. Start the program.
2. Include the standard input-output header file <stdio.h>.
3. Declare variables for student name, subject, roll number, and marks.
4. Use scanf() to get input values from the user.
5. Declare a file pointer FILE *fp.
6. Open the file "student.txt" in write mode ("w") using fopen().
7. Write the student name and roll number into the file using fprintf().
8. Close the file using fclose().
9. Reopen the same file in append mode ("a") using fopen().
10. Append the subject and marks details into the file using fprintf().
11. Close the file again using fclose().
12. Open the file in read mode ("r") using fopen().
13. Declare a character variable ch.
14. Use fgetc() inside a while loop to read each character from the file until EOF is reached.
15. Display the file contents on the screen using printf().
16. Close the file using fclose().
17. Stop the program.

Program:

```c
#include <stdio.h>
int main() {
    char name[100], subject[100];
    int roll, marks;
    scanf("%s", name);
    scanf("%d", &roll);
    scanf("%s", subject);
    scanf("%d", &marks);
    FILE *fp;
    fp = fopen("student.txt", "w");
    fprintf(fp, "Student Name: %s\n", name);
    fprintf(fp, "Roll Number: %d\n", roll);
    fclose(fp);
    fp = fopen("student.txt", "a");
    fprintf(fp, "Subject: %s\n", subject);
    fprintf(fp, "Marks: %d\n", marks);
    fclose(fp);
    fp = fopen("student.txt", "r");
    char ch;
    while ((ch = fgetc(fp)) != EOF) {
        printf("%c", ch);
    }
    fclose(fp);
    return 0;
}
```

Output:

```
Student Name: Arun
Roll Number: 25
Subject: Physics
Marks: 88
```

Result:

Thus the C program to create, write, append, and read student details using file handling was written, compiled, and executed successfully, and the output was verified.